Table des matières

Labo 03 OLAP — Écrire dans l'entrepôt 1
Le terrain, en une minute 2
Partie 1 — Le bac à sable 2
Partie 2 — Créer une table et ses types 3
Partie 3 — Les contraintes 5
Partie 4 — INSERT 7
Partie 5 — UPDATE et DELETE 9
Partie 6 — MERGE 11
Partie 7 — Les vues 14
Partie 8 — Ce qui s'annule 16
Nettoyage 17
Et ensuite ? 17

Labo 03 OLAP — Écrire dans l'entrepôt

Laboratoire du Cours 03, voie OLAP · DuckDB + AdventureWorks DW · Formation SQL avancé

On pratique l'écriture sur l'entrepôt AdventureWorks DW, dans la ligne de commande DuckDB. Depuis `Projet_SQL/`, ouvrez l'entrepôt par `duckdb adventureworksdw.duckdb`, puis posez le schéma par `USE dbo;`. Tout ce que le labo crée porte le préfixe `LABO03_`, et les tables d'AdventureWorks ne servent qu'à être lues. Comptez une heure trois quarts.

Les deux voies — Ce labo a un jumeau, `Labo_03_OLTP_SQL.md`, qui fait les mêmes gestes sur PostgreSQL. Faites celui-ci d'abord. Les deux moteurs appliquent les mêmes contraintes, mais pas les mêmes types ni les mêmes garde-fous : plusieurs commandes qui passent ici en silence y sont refusées, et l'inverse arrive aussi. C'est tout l'intérêt de les faire deux fois.

Une commande à la fois — Plusieurs étapes contiennent des erreurs voulues. Collez les commandes une par une : une erreur n'annule que la commande fautive, et la session continue. Quand une étape ouvre une transaction avec `BEGIN`, allez jusqu'au `ROLLBACK` indiqué avant de passer à la suite. Dans une transaction, la moindre erreur la met en échec, et DuckDB refuse ensuite toute commande (`Current transaction is aborted (please ROLLBACK)`) jusqu'au `ROLLBACK`.

© 2026 iAlexMG. Tous droits réservés.

Le terrain, en une minute

Table	Ce qu'elle contient	Grain	Ce qui servira ici
`DimCustomer`	les clients des ventes en ligne	un client, 18 484 lignes	la source qu'on copie : les 506 clients arrivés en janvier 2014

```
USE dbo;
.changes on
```

La seconde ligne n'est pas du SQL : c'est une commande de la ligne de commande, qui commence par un point comme le `.quit` du Cours 00. Elle fait suivre chaque commande d'une ligne `changes: N`, le nombre de lignes que la dernière écriture a touchées. C'est le compte rendu du moteur, et ce labo le lit souvent. Après un `SELECT`, la ligne répète le compte de l'écriture précédente : ne la lisez qu'après un `INSERT`, un `UPDATE`, un `DELETE` ou un `MERGE`.

Une seule table lue, et une douzaine d'objets de travail créés au fil du labo, des tables, des vues et une séquence, tous préfixés `LABO03_` et supprimés dans la section Nettoyage.

Partie 1 — Le bac à sable

1.1 — Copier un extrait

Théorie : `Theorie_03_SQL.md`, sections 1 et 2.

Objectif : fabriquer une table de travail avec les clients arrivés en janvier 2014.

```
CREATE OR REPLACE TABLE LABO03_CLIENT_BRUT AS
SELECT CustomerKey, FirstName, MiddleName, LastName, EmailAddress,
       YearlyIncome, HouseOwnerFlag, CommuteDistance, DateFirstPurchase
FROM DimCustomer
WHERE DateFirstPurchase >= '2014-01-01';

SELECT COUNT(*) AS clients, MIN(CustomerKey) AS premier, MAX(CustomerKey) AS dernier
FROM LABO03_CLIENT_BRUT;
```

Observez. 506 clients, numérotés de 11 349 à 29 471. Regardez maintenant la structure de la copie. `DESCRIBE` rend un tableau comme un autre, qu'on peut interroger : on n'en garde ici que les cinq colonnes qui servent.

```
SELECT column_name, column_type, "null", "key", "default"
FROM (DESCRIBE LABO03_CLIENT_BRUT);
```

Observez. Les types sont ceux de `DimCustomer` : `DECIMAL(19,4)` pour le revenu, `DATE` pour la date du premier achat, `VARCHAR` sans longueur pour le texte. La colonne `null` dit `YES` sur les neuf lignes, et `key` comme `default` valent `NULL` : toutes les colonnes acceptent `NULL`, aucune n'a de valeur par défaut, et aucune clé

© 2026 iAlexMG. Tous droits réservés.

n'est déclarée. `DimCustomer` n'en avait pas non plus, le script du Cours 00 n'ayant déclaré aucune contrainte. La copie ne perd donc rien ici, mais elle n'aurait rien gardé de toute façon : un `CREATE TABLE … AS` reprend les noms et les types, jamais une clé, un `NOT NULL` ou une valeur par défaut.

1.2 — `CREATE OR REPLACE`, la suppression déguisée

Théorie : `Theorie_03_SQL.md`, section 2.

Objectif : voir ce que fait une recréation.

Relancez le `CREATE OR REPLACE TABLE` de la sous-partie 1.1, puis recomptez.

Observez. Aucune question, aucun avertissement, et toujours 506 lignes. La table a été supprimée et recréée d'un coup. Dans un bac à sable, c'est commode ; sur une table que d'autres lisent, c'est un `DROP TABLE` sous un autre nom. DuckDB ne garde aucune version passée d'une table : sauf dans une transaction ouverte avant la commande (partie 8), l'ancienne est perdue.

Portabilité — PostgreSQL n'a pas `CREATE OR REPLACE TABLE` : on y écrit un `DROP TABLE`, puis un `CREATE TABLE`. Snowflake a la commande, et un filet que DuckDB n'a pas : son voyage dans le temps, présenté au Cours 11, garde l'ancienne version de la table pendant une durée de rétention.

Partie 2 — Créer une table et ses types

2.1 — Ce que le type fait à la valeur

Théorie : `Theorie_03_SQL.md`, section 2.

Objectif : voir, valeur par valeur, ce qu'un type refuse et ce qu'il transforme.

```sql
CREATE OR REPLACE TABLE LABO03_ESSAI_TYPE (
    Montant  DECIMAL(6,2),
    Code     VARCHAR(3),
    Quantite SMALLINT
);

INSERT INTO LABO03_ESSAI_TYPE (Montant)  VALUES (12.345);
INSERT INTO LABO03_ESSAI_TYPE (Montant)  VALUES (12345.6);
INSERT INTO LABO03_ESSAI_TYPE (Code)     VALUES ('ABCD');
INSERT INTO LABO03_ESSAI_TYPE (Quantite) VALUES (40000);
INSERT INTO LABO03_ESSAI_TYPE (Quantite) VALUES (12.6);
```

Observez. Deux refus, dont les messages disent la cause : `Casting value "12345.6" to type DECIMAL(6,2) failed: value is out of range!`, puis, pour 40 000, `Type INT32 with value 40000 can't be cast because the value is out of range for the destination type INT16.` `SMALLINT` est un entier sur deux octets, `INT16` dans le vocabulaire de DuckDB, qui s'arrête à 32 767. Les trois autres commandes passent. Regardez ce qu'elles ont rangé :

© 2026 iAlexMG. Tous droits réservés.

```sql
SELECT Montant, Code, LENGTH(Code) AS longueur, Quantite
FROM LABO03_ESSAI_TYPE;

SELECT column_name, column_type
FROM (DESCRIBE LABO03_ESSAI_TYPE);
```

Observez. 12,345 est devenu 12,35, et 12,6 est devenu 13 : deux arrondis sans un mot. Et `ABCD`, quatre caractères, est entré en entier dans une colonne déclarée `VARCHAR(3)`. Le `DESCRIBE` dit pourquoi : la colonne est `VARCHAR`, tout court. DuckDB accepte qu'on écrive une longueur, puis l'oublie, et ne la vérifie jamais. Sur ce moteur, la longueur d'un texte n'est pas une contrainte.

Portabilité — Le nom d'un type ne dit pas ce qu'il garantit, et chaque moteur a sa lecture. PostgreSQL refuse `ABCD` dans un `VARCHAR(3)`, le labo jumeau vous le montrera. Snowflake, d'après sa documentation, vérifie la longueur d'un `VARCHAR` comme PostgreSQL, mais fait de `SMALLINT`, `INT` et `BIGINT` trois noms du même type, `NUMBER(38,0)` : le 40 000 y entrerait sans erreur.

2.2 — Une table faite pour durer

Théorie : `Theorie_03_SQL.md`, sections 2 et 3.

Objectif : créer la table des clients de la campagne de bienvenue, avec ses contraintes, puis la remplir.

```sql
CREATE OR REPLACE TABLE LABO03_CLIENT (
    CustomerKey       INTEGER       PRIMARY KEY,
    FirstName         VARCHAR       NOT NULL,
    MiddleName        VARCHAR,
    LastName          VARCHAR       NOT NULL,
    EmailAddress      VARCHAR       UNIQUE,
    YearlyIncome      DECIMAL(19,4),
    HouseOwnerFlag    VARCHAR,
    CommuteDistance   VARCHAR,
    DateFirstPurchase DATE          NOT NULL,
    Segment           VARCHAR       DEFAULT 'nouveau',
    ChargeLe          TIMESTAMP     DEFAULT CURRENT_TIMESTAMP
);

INSERT INTO LABO03_CLIENT (CustomerKey, FirstName, MiddleName, LastName, EmailAddress,
                           YearlyIncome, HouseOwnerFlag, CommuteDistance, DateFirstPurchase)
SELECT CustomerKey, FirstName, MiddleName, LastName, EmailAddress,
       YearlyIncome, HouseOwnerFlag, CommuteDistance, DateFirstPurchase
FROM LABO03_CLIENT_BRUT;
```

Observez. `changes: 506`. Les textes sont déclarés `VARCHAR` sans longueur : la sous-partie 2.1 vient de montrer que DuckDB n'en vérifierait aucune, et une limite écrite que personne ne fait respecter trompe le lecteur du code. Les deux colonnes que l'`INSERT` ne nommait pas ont reçu leur valeur par défaut :

© 2026 iAlexMG. Tous droits réservés.

```sql
SELECT Segment, COUNT(*) AS clients, MIN(ChargeLe) AS charge_le
FROM LABO03_CLIENT
GROUP BY ALL;
```

Observez. Une seule ligne : 506 clients au segment « nouveau », tous chargés à l'instant. `CURRENT_TIMESTAMP` s'écrit ici sans parenthèses, comme le veut la norme ; avec, DuckDB répond `Scalar Function with name current_timestamp does not exist!`.

Partie 3 — Les contraintes

3.1 — Violer chaque contrainte, une à la fois

Théorie : `Theorie_03_SQL.md`, section 3.

Objectif : lire le message de chaque contrainte, et constater que rien n'est écrit.

Mindy Luo, cliente 11349, est déjà dans la table. Tentez de l'inscrire une seconde fois, puis d'inscrire quelqu'un d'autre avec son adresse, puis une cliente sans prénom :

```sql
-- clé primaire : le numéro 11349 existe déjà
INSERT INTO LABO03_CLIENT (CustomerKey, FirstName, LastName, EmailAddress, DateFirstPurchase)
VALUES (11349, 'Mindy', 'Luo', 'mindy10@adventure-works.com', '2014-01-13');

-- UNIQUE : un autre numéro, la même adresse
INSERT INTO LABO03_CLIENT (CustomerKey, FirstName, LastName, EmailAddress, DateFirstPurchase)
VALUES (99998, 'Mindy', 'Luo', 'mindy10@adventure-works.com', '2014-01-13');

-- NOT NULL : pas de prénom
INSERT INTO LABO03_CLIENT (CustomerKey, FirstName, LastName, DateFirstPurchase)
VALUES (99999, NULL, 'Luo', '2014-01-13');
```

Observez. Trois refus, trois messages : `Duplicate key "CustomerKey: 11349" violates primary key constraint`, `Duplicate key "EmailAddress: mindy10@adventure-works.com" violates unique constraint`, puis `NOT NULL constraint failed: LABO03_CLIENT.FirstName`. Chacun nomme la colonne et la valeur fautives, ce qui vaut mieux qu'un nom de contrainte à déchiffrer. Au tour du `CHECK` :

```sql
CREATE OR REPLACE TABLE LABO03_CHECK (x INTEGER CHECK (x > 0));
INSERT INTO LABO03_CHECK VALUES (0);
INSERT INTO LABO03_CHECK VALUES (NULL);
SELECT * FROM LABO03_CHECK;
```

Observez. La table est créée, le 0 est refusé (`CHECK constraint failed on table LABO03_CHECK with expression CHECK((x > 0))`), et le `NULL` passe : la condition n'est ni vraie ni fausse pour lui, et un `CHECK` ne refuse que le faux. Pour interdire le `NULL`, il faut un `NOT NULL` en plus. Recomptez enfin, avec la requête de contrôle de la théorie :

© 2026 iAlexMG. Tous droits réservés.

```sql
SELECT CustomerKey, COUNT(*) AS lignes
FROM LABO03_CLIENT
GROUP BY CustomerKey
HAVING COUNT(*) > 1;
```

Observez. Aucune ligne : les commandes refusées n'ont rien écrit. DuckDB applique les cinq contraintes, comme PostgreSQL. L'entrepôt de la formation, lui, n'en a aucune, puisque le script du Cours 00 n'en déclare pas. Lancez la même requête sur sa table des clients :

```sql
SELECT CustomerKey, COUNT(*) AS lignes
FROM DimCustomer
GROUP BY CustomerKey
HAVING COUNT(*) > 1;
```

Observez. Aucune ligne non plus, mais ce n'est pas la même chose : ici, c'est un constat, pas une garantie. Le prochain chargement pourrait y faire entrer un doublon, et rien ne l'arrêterait.

Portabilité — Tous les entrepôts ne font pas le travail de DuckDB. D'après sa documentation, Snowflake enregistre une clé primaire, une contrainte `UNIQUE` ou une clé étrangère sur une table ordinaire sans les appliquer ; seuls `NOT NULL` et `CHECK` y sont vérifiés. D'autres entrepôts font le même choix, et c'est dans la documentation du moteur qu'on le vérifie. Sur ces moteurs, le doublon de Mindy Luo entrerait sans un mot, et seule la requête de contrôle ci-dessus le révélerait. C'est elle qu'il faut garder en réflexe, et c'est exactement ce que fait le test `unique` de dbt, dans la formation Analytics Engineering.

3.2 — La clé étrangère, et le compteur à part

Théorie : `Theorie_03_SQL.md`, section 3.

Objectif : créer le journal des courriels de bienvenue, lié aux clients.

```sql
CREATE SEQUENCE LABO03_ENVOI_SEQ START 1;

CREATE OR REPLACE TABLE LABO03_ENVOI (
    EnvoiKey    INTEGER   PRIMARY KEY DEFAULT NEXTVAL('LABO03_ENVOI_SEQ'),
    CustomerKey INTEGER   NOT NULL REFERENCES LABO03_CLIENT (CustomerKey),
    Canal       VARCHAR   NOT NULL DEFAULT 'courriel',
    EnvoyeLe    TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
);

INSERT INTO LABO03_ENVOI (EnvoiKey, CustomerKey)
VALUES (999, 99);
```

Observez. La table est créée, et l'envoi au client 99, qui n'existe pas, est refusé : `Violates foreign key constraint because key "CustomerKey: 99" does not exist in the referenced table`. La clé étrangère protège le journal contre les clients fantômes ; la partie 5 la verra défendre aussi l'autre côté.

© 2026 iAlexMG. Tous droits réservés.

Le numéro d'envoi, lui, vient d'un objet à part. DuckDB n'a pas de colonne d'identité : il refuse `GENERATED BY DEFAULT AS IDENTITY` (`Constraint not implemented!`), comme `AUTOINCREMENT` et `SERIAL`. Le compteur est une séquence, créée d'abord, que la valeur par défaut de `EnvoiKey` appelle par `NEXTVAL` à chaque ligne insérée sans numéro. La colonne d'identité de PostgreSQL repose sur le même mécanisme, caché derrière la colonne.

Portabilité — Snowflake déclare le compteur sur la colonne, par `AUTOINCREMENT` ou `IDENTITY`, et, d'après sa documentation, enregistre la clé étrangère sans l'appliquer : l'envoi au client 99 y entrerait.

Partie 4 — `INSERT`

4.1 — Nommer les colonnes, et laisser le moteur numéroter

Théorie : `Theorie_03_SQL.md`, section 4.

Objectif : charger l'historique des envois avec ses numéros, puis ajouter la campagne du jour.

```sql
INSERT INTO LABO03_ENVOI (EnvoiKey, CustomerKey, EnvoyeLe)
VALUES (1, 11383, '2014-01-02 09:00'),
       (2, 11435, '2014-01-05 09:00'),
       (3, 11441, '2014-01-09 09:00');

INSERT INTO LABO03_ENVOI (CustomerKey)
SELECT CustomerKey
FROM LABO03_CLIENT
WHERE YearlyIncome >= 100000;
```

Observez. `changes: 3`, puis une erreur : `Duplicate key "EnvoiKey: 1" violates primary key constraint`. La campagne visait les clients au revenu d'au moins 100 000 $, et aucun n'a reçu d'envoi. Dans le second `INSERT`, ni le numéro, ni le canal, ni l'heure ne sont fournis : la séquence et les valeurs par défaut devaient s'en charger, et c'est la séquence qui a buté.

4.2 — Le compteur aveugle

Théorie : `Theorie_03_SQL.md`, section 4.

Objectif : voir où en est le compteur, puis le recaler au-delà des numéros chargés.

```sql
SELECT start_value, last_value
FROM DUCKDB_SEQUENCES()
WHERE sequence_name = 'LABO03_ENVOI_SEQ';
```

Observez. Une séquence partie de 1, et un dernier numéro tiré de 67. La séquence a commencé à 1 sans regarder les trois envois chargés avec leurs numéros, et la clé primaire a arrêté la commande au premier doublon. Mais la commande avait déjà tiré un numéro pour chacun des 67 clients visés, et ces 67 numéros sont

© 2026 iAlexMG. Tous droits réservés.

perdus : une séquence ne recule jamais, même quand la commande qui l'a fait avancer échoue. Le bon geste est de recaler le compteur au-delà du plus grand numéro chargé, et DuckDB ne le permet pas encore :

```
ALTER SEQUENCE LABO03_ENVOI_SEQ RESTART WITH 4;
```

Observez. `ALTER SEQUENCE option not supported yet!`, et il n'y a pas de `setval` non plus. Une séquence de DuckDB se fixe à sa création, et celle dont une table se sert ne se supprime pas. Le recalage passe donc par un détour : détacher la séquence de la colonne, la recréer au-delà du plus grand numéro, puis la rattacher.

```
SELECT MAX(EnvoiKey) AS plus_grand FROM LABO03_ENVOI;

ALTER TABLE LABO03_ENVOI ALTER COLUMN EnvoiKey DROP DEFAULT;
DROP SEQUENCE LABO03_ENVOI_SEQ;
CREATE SEQUENCE LABO03_ENVOI_SEQ START 4;
ALTER TABLE LABO03_ENVOI ALTER COLUMN EnvoiKey SET DEFAULT NEXTVAL('LABO03_ENVOI_SEQ');
```

Observez. Le plus grand numéro chargé est 3, d'où le `START 4`. DuckDB veut là un nombre écrit en toutes lettres, et refuse une sous-requête : le plus grand numéro se lit d'abord, puis se recopie. Relancez maintenant la campagne :

```
INSERT INTO LABO03_ENVOI (CustomerKey)
SELECT CustomerKey
FROM LABO03_CLIENT
WHERE YearlyIncome >= 100000;

SELECT COUNT(*) AS envois, MIN(EnvoiKey) AS premier, MAX(EnvoiKey) AS dernier
FROM LABO03_ENVOI
WHERE EnvoiKey > 3;
```

Observez. `changes: 67`, et 67 envois numérotés de 4 à 70, à la suite de l'historique. Le plus simple reste de ne pas se mettre dans ce cas : charger l'historique d'abord, et créer la séquence ensuite, au-delà de son plus grand numéro. Sur PostgreSQL, le compteur se recale d'une commande, et le labo jumeau le fait.

4.3 — Avec `RETURNING`

Théorie : `Theorie_03_SQL.md`, section 4.

Objectif : ajouter un envoi, et récupérer le numéro que le moteur lui donne.

```
INSERT INTO LABO03_ENVOI (CustomerKey, Canal)
VALUES (11441, 'sms')
RETURNING EnvoiKey, CustomerKey, Canal;
```


© 2026 iAlexMG. Tous droits réservés.

Observez. Une ligne revient : l'envoi 71, à la cliente 11441, Erika Gomez, par `sms`. `RETURNING` rend les colonnes de la ligne telle que le moteur l'a écrite, numéro généré compris. C'est le seul moyen sûr de connaître ce numéro : le réflexe du « plus grand numéro » se trompe dès que quelqu'un a écrit un numéro à la main, ou qu'une autre session insère en même temps.

Portabilité — PostgreSQL a le même `RETURNING`. Snowflake ne l'a pas : on y retrouve les lignes qu'un `INSERT` a créées par ce qu'on a inséré, ici le client et l'heure d'envoi, jamais par un numéro qu'on devine.

4.4 — L'`INSERT` sans liste de colonnes

Théorie : `Theorie_03_SQL.md`, section 4.

Objectif : voir DuckDB refuser une liste de valeurs incomplète.

```
INSERT INTO LABO03_ENVOI
VALUES (1000, 11349);
```

Observez. `table LABO03_ENVOI has 4 columns but 2 values were supplied` : la table a quatre colonnes, la commande n'en fournit que deux, et DuckDB refuse de deviner, même si les deux autres ont une valeur par défaut. PostgreSQL accepte la même commande et complète par les défauts ; le labo jumeau vous montrera surtout ce qu'il fait d'une liste complète dont deux valeurs sont inversées. DuckDB offre une troisième écriture, `INSERT INTO … BY NAME SELECT …`, qui range chaque colonne du `SELECT` dans la colonne de même nom, dans n'importe quel ordre, et laisse les autres à leur valeur par défaut.

Partie 5 — `UPDATE` et `DELETE`

5.1 — Supprimer une ligne, et seulement elle

Théorie : `Theorie_03_SQL.md`, section 5.

Objectif : retirer l'envoi par SMS, puis voir la clé étrangère défendre un client.

```
SELECT COUNT(*) FROM LABO03_ENVOI WHERE Canal = 'sms';

DELETE FROM LABO03_ENVOI
WHERE Canal = 'sms';
```

Observez. Une ligne comptée, `changes: 1`. Un `WHERE` sur la seule cliente aurait emporté aussi son envoi du 9 janvier : il fallait une condition qui distingue les deux, ici le canal. Tentez maintenant de retirer une cliente :

```
DELETE FROM LABO03_CLIENT
WHERE CustomerKey = 11383;
```


© 2026 iAlexMG. Tous droits réservés.

Observez. Refus : `Violates foreign key constraint because key "CustomerKey: 11383" is still referenced by a foreign key in a different table`. Marie Gill a reçu l'envoi 1, et la clé étrangère qui interdisait un envoi sans client interdit aussi de retirer un client sous ses envois. Pour la supprimer, il faudrait retirer ses envois d'abord, les enfants avant le parent.

5.2 — Compter, écrire, vérifier

Théorie : `Theorie_03_SQL.md`, sections 5 et 8.

Objectif : marquer les clients aisés, puis rattraper l'oubli du `WHERE`.

```
SELECT COUNT(*) FROM LABO03_CLIENT WHERE YearlyIncome >= 100000;

UPDATE LABO03_CLIENT
SET Segment = 'aisé'
WHERE YearlyIncome >= 100000;
```

Observez. 67 lignes comptées, `changes: 67` : le compte rendu confirme le compte. Refaites maintenant l'erreur classique, dans une transaction :

```
BEGIN;
UPDATE LABO03_CLIENT SET Segment = 'aisé';
ROLLBACK;

SELECT Segment, COUNT(*) AS clients
FROM LABO03_CLIENT
GROUP BY ALL
ORDER BY Segment;
```

Observez. L'`UPDATE` sans `WHERE` annonce `changes: 506`, puis le `ROLLBACK` les rend : 67 clients « aisé », 439 « nouveau ». Sans le `BEGIN`, la modification aurait été validée à la seconde où elle a réussi.

5.3 — `UPDATE … FROM` et la source en double

Théorie : `Theorie_03_SQL.md`, section 5.

Objectif : corriger une adresse à partir d'une liste qui en contient deux pour la même cliente.

```
UPDATE LABO03_CLIENT AS c
SET EmailAddress = s.Courriel
FROM (SELECT 11349 AS CustomerKey, 'mindy.luo@exemple.test' AS Courriel
      UNION ALL
      SELECT 11349, 'm.luo@exemple.test') AS s
WHERE c.CustomerKey = s.CustomerKey;

SELECT CustomerKey, FirstName, LastName, EmailAddress
```


© 2026 iAlexMG. Tous droits réservés.

```
FROM LABO03_CLIENT
WHERE CustomerKey = 11349;
```

Observez. Aucune erreur, et un compte rendu qui dit `changes: 2` pour une seule cliente : DuckDB a compté chaque correspondance de la source, pas chaque ligne de la table. Mindy Luo a reçu l'une des deux adresses, sans que le moteur dise laquelle ni pourquoi. Deux lancements du même labo nous ont donné les deux : `mindy.luo@exemple.test` une fois, `m.luo@exemple.test` l'autre. Ici, le compte rendu trahit le défaut, à condition de le comparer au nombre de clientes visées.

Tentez la même correction, sans doublon cette fois, sur Marie Gill :

```
UPDATE LABO03_CLIENT
SET EmailAddress = 'marie.gill@exemple.test'
WHERE CustomerKey = 11383;
```

Observez. Refus, et un message inattendu : la clé étrangère, `key "CustomerKey: 11383" is still referenced`, alors que la commande ne touche pas à la clé. Le message renvoie lui-même aux limites de DuckDB, et c'en est une, décrite dans sa documentation : une modification qui touche une colonne indexée, ici l'adresse que `UNIQUE` indexe, s'y exécute comme une suppression suivie d'une insertion, et la suppression d'une cliente qui a des envois est interdite. Le segment et le revenu, qui n'ont pas d'index, se modifient sans encombre, comme en 5.2. Sur DuckDB, une clé étrangère fige donc les colonnes indexées de la table qu'elle vise.

5.4 — `TRUNCATE`

Théorie : `Theorie_03_SQL.md`, sections 3 et 5.

Objectif : vider la table d'essai des types, puis tenter de vider celle des clients.

```
TRUNCATE TABLE LABO03_ESSAI_TYPE;
SELECT COUNT(*) FROM LABO03_ESSAI_TYPE;
TRUNCATE TABLE LABO03_CLIENT;
```

Observez. Zéro ligne dans la table d'essai. Sur `LABO03_CLIENT`, le même refus qu'en 5.1, pour le premier client qui a un envoi : `TRUNCATE` ne fait pas exception à la clé étrangère, et le journal des envois ne pointera jamais vers des clients disparus.

Partie 6 — `MERGE`

6.1 — Fusionner une mise à jour

Théorie : `Theorie_03_SQL.md`, section 6.

Objectif : appliquer un fichier de mise à jour qui contient un client connu et un client oublié.

© 2026 iAlexMG. Tous droits réservés.

Le service à la clientèle envoie deux lignes : Marie Gill, 11383, dont le revenu est passé à 40 000 $, et Dawn Luo, 13804, arrivée le 31 décembre 2013 et restée hors de l'extrait.

```
CREATE OR REPLACE TABLE LABO03_CLIENT_MAJ AS
SELECT CustomerKey, FirstName, MiddleName, LastName, EmailAddress,
       IF(CustomerKey = 11383, 40000, YearlyIncome) AS YearlyIncome,
       HouseOwnerFlag, CommuteDistance, DateFirstPurchase
FROM DimCustomer
WHERE CustomerKey IN (11383, 13804);

MERGE INTO LABO03_CLIENT AS c
USING LABO03_CLIENT_MAJ AS m
   ON m.CustomerKey = c.CustomerKey
WHEN MATCHED THEN
    UPDATE SET YearlyIncome = m.YearlyIncome
WHEN NOT MATCHED THEN
    INSERT (CustomerKey, FirstName, MiddleName, LastName, EmailAddress,
            YearlyIncome, HouseOwnerFlag, CommuteDistance, DateFirstPurchase)
    VALUES (m.CustomerKey, m.FirstName, m.MiddleName, m.LastName, m.EmailAddress,
            m.YearlyIncome, m.HouseOwnerFlag, m.CommuteDistance, m.DateFirstPurchase);

SELECT CustomerKey, FirstName, LastName, YearlyIncome, Segment
FROM LABO03_CLIENT
WHERE CustomerKey IN (11383, 13804)
ORDER BY CustomerKey;
```

Observez. `changes: 2`, sans dire lesquels, et la requête départage : Marie Gill a son nouveau revenu, 40 000 $, et Dawn Luo est entrée avec le segment « nouveau », par défaut.

6.2 — Rejouer

Théorie : `Theorie_03_SQL.md`, section 6.

Objectif : vérifier qu'un chargement relancé ne crée pas de doublon.

Relancez exactement le même `MERGE`, puis comptez :

```
SELECT COUNT(*) FROM LABO03_CLIENT;
```

Observez. `changes: 2` encore, et 507 clients : les 506 du départ plus Dawn Luo, une seule fois. Au second passage, les deux lignes de la source tombent dans `WHEN MATCHED`. Un `INSERT … SELECT` relancé par erreur aurait buté ici sur la clé primaire ; sur un entrepôt qui ne l'applique pas, il aurait inscrit Dawn Luo deux fois.

© 2026 iAlexMG. Tous droits réservés.

6.3 — La source en double

Théorie : `Theorie_03_SQL.md`, section 6.

Objectif : voir si `MERGE` refuse ce que l'`UPDATE … FROM` de la partie 5 acceptait.

Le service à la clientèle renvoie la ligne de Marie Gill, avec son ancien revenu, sans retirer la nouvelle. Ajoutez-la à la source, puis relancez la fusion :

```
INSERT INTO LABO03_CLIENT_MAJ
SELECT CustomerKey, FirstName, MiddleName, LastName, EmailAddress,
       YearlyIncome, HouseOwnerFlag, CommuteDistance, DateFirstPurchase
FROM DimCustomer
WHERE CustomerKey = 11383;

MERGE INTO LABO03_CLIENT AS c
USING LABO03_CLIENT_MAJ AS m
   ON m.CustomerKey = c.CustomerKey
WHEN MATCHED THEN
    UPDATE SET YearlyIncome = m.YearlyIncome
WHEN NOT MATCHED THEN
    INSERT (CustomerKey, FirstName, MiddleName, LastName, EmailAddress,
            YearlyIncome, HouseOwnerFlag, CommuteDistance, DateFirstPurchase)
    VALUES (m.CustomerKey, m.FirstName, m.MiddleName, m.LastName, m.EmailAddress,
            m.YearlyIncome, m.HouseOwnerFlag, m.CommuteDistance, m.DateFirstPurchase);

SELECT CustomerKey, YearlyIncome
FROM LABO03_CLIENT
WHERE CustomerKey = 11383;
```

Observez. Aucune erreur, et `changes: 3` pour deux clients. La source propose deux revenus pour Marie Gill, 40 000 $ et 30 000 $; DuckDB a compté les deux mises à jour, et une seule valeur reste. Nos essais ont tous gardé 30 000 $, mais rien ne garantit que ce sera la même chez vous. C'est le défaut de l'`UPDATE … FROM` de la sous-partie 5.3, et la même conclusion : sur DuckDB, `MERGE` ne protège pas contre une source en double. La parade est la requête de contrôle, lancée sur la source avant la fusion :

```
SELECT CustomerKey, COUNT(*) AS lignes
FROM LABO03_CLIENT_MAJ
GROUP BY CustomerKey
HAVING COUNT(*) > 1;
```

Observez. Une ligne, le client 11383, deux fois : la fusion n'aurait pas dû partir.

Piège — Sur ce point, les moteurs divergent, et DuckDB est le plus permissif. PostgreSQL refuse un `MERGE` dont la source propose deux lignes pour une même ligne de la cible, le labo jumeau vous le montrera. Snowflake le refuse aussi d'après sa documentation, tant que son paramètre

© 2026 iAlexMG. Tous droits réservés.

`ERROR_ON_NONDETERMINISTIC_MERGE` garde sa valeur par défaut. DuckDB, mesuré en version 1.5.5, applique les deux lignes sans un mot. Une fusion qui passe d'un moteur protecteur à DuckDB perd ce garde-fou sans la moindre erreur pour le signaler : contrôlez la source vous-même, partout.

6.4 — L'upsert par `ON CONFLICT`

Théorie : `Theorie_03_SQL.md`, section 6.

Objectif : écrire une mise à jour par la forme courte, et rendre à Marie Gill son revenu d'origine.

```
INSERT INTO LABO03_CLIENT (CustomerKey, FirstName, LastName, YearlyIncome, DateFirstPurchase)
VALUES (11383, 'Marie', 'Gill', 30000, '2014-01-01')
ON CONFLICT (CustomerKey) DO UPDATE
    SET YearlyIncome = EXCLUDED.YearlyIncome;

SELECT CustomerKey, YearlyIncome
FROM LABO03_CLIENT
WHERE CustomerKey = 11383;
```

Observez. 30 000 $, quel qu'ait été le résultat de la sous-partie 6.3. La ligne proposée est entrée en conflit avec la clé primaire, et la branche `DO UPDATE` a pris le relais, `EXCLUDED` désignant la ligne refusée. DuckDB a repris cette forme de PostgreSQL : sur les deux terrains, l'upsert s'écrit par `MERGE` ou par `ON CONFLICT`.

Partie 7 — Les vues

7.1 — Une vue suit les données, une colonne calculée non

Théorie : `Theorie_03_SQL.md`, section 7.

Objectif : comparer une vue et une colonne remplie par un `UPDATE`.

```
CREATE OR REPLACE VIEW LABO03_CLIENT_AISE AS
SELECT CustomerKey, FirstName, LastName, YearlyIncome
FROM LABO03_CLIENT
WHERE YearlyIncome >= 100000;

SELECT COUNT(*) FROM LABO03_CLIENT_AISE;
```

Observez. 67 clients. Robin Romero, 11435, déclare maintenant 90 000 $:

```
UPDATE LABO03_CLIENT SET YearlyIncome = 90000 WHERE CustomerKey = 11435;

SELECT COUNT(*) FROM LABO03_CLIENT_AISE;

SELECT COUNT(*) AS encore_marques_aises
```


© 2026 iAlexMG. Tous droits réservés.

```
FROM LABO03_CLIENT
WHERE Segment = 'aisé';
```

Observez. La vue compte 66 clients, sans qu'on l'ait touchée. La colonne `Segment`, elle, en marque toujours 67 : elle a été calculée une fois, en 5.2, et rien ne la recalcule. C'est tout le choix entre une vue et une table de travail : l'une relit la source à chaque fois, l'autre garde une photo, qui vieillit.

7.2 — La vue `SELECT *` après un ajout de colonne

Théorie : `Theorie_03_SQL.md`, section 7.

Objectif : voir ce qu'une modification de table fait à une vue écrite avec l'étoile.

```
CREATE OR REPLACE VIEW LABO03_CLIENT_TOUT AS
SELECT * FROM LABO03_CLIENT;

SELECT COUNT(*) AS colonnes FROM (DESCRIBE LABO03_CLIENT_TOUT);

ALTER TABLE LABO03_CLIENT ADD COLUMN Langue VARCHAR DEFAULT 'en';

SELECT COUNT(*) AS colonnes FROM (DESCRIBE LABO03_CLIENT_TOUT);

SELECT Langue, COUNT(*) AS clients
FROM LABO03_CLIENT_TOUT
GROUP BY ALL;
```

Observez. Onze colonnes, puis douze, et 507 clients dont la langue vaut `en` : la vue a pris la nouvelle colonne sans qu'on la touche. DuckDB garde le texte de la requête et relit l'étoile à chaque interrogation, dans les deux sens : une colonne retirée de la table disparaîtrait de la vue tout aussi discrètement. Un tableau de bord branché sur une vue en `SELECT *` voit sa forme changer au gré de la table ; une vue qui nomme ses colonnes n'a pas ce problème.

Portabilité — PostgreSQL fait l'inverse : il remplace l'étoile par la liste des colonnes au moment de créer la vue, et cette liste ne bouge plus, le labo jumeau vous le montrera. Deux moteurs, deux comportements pour la même vue : n'en supposez aucun, nommez les colonnes.

7.3 — Supprimer une table sous une vue

Théorie : `Theorie_03_SQL.md`, section 7.

Objectif : voir si DuckDB protège les dépendants d'une table.

```
CREATE OR REPLACE TABLE LABO03_JETABLE AS SELECT 1 AS x;
CREATE OR REPLACE VIEW LABO03_VUE_JETABLE AS SELECT x FROM LABO03_JETABLE;

DROP TABLE LABO03_JETABLE;
SELECT * FROM LABO03_VUE_JETABLE;
```


© 2026 iAlexMG. Tous droits réservés.

Observez. Le `DROP TABLE` passe sans un mot, et c'est la vue qui échoue ensuite : `Table with name LABO03_JETABLE does not exist!`. DuckDB ne tient pas le compte de ce qui dépend d'une table par une vue ; PostgreSQL aurait refusé la suppression et listé la vue. Il le tient pour la clé étrangère, en revanche :

```
DROP TABLE LABO03_CLIENT;
```

Observez. Refus : `Could not drop the table because this table is main key table of the table "LABO03_ENVOI"`. Sur l'entrepôt, un nettoyage trop large casse les vues des autres, et on ne l'apprend qu'à leur prochaine requête ; seule une clé étrangère l'arrête.

Partie 8 — Ce qui s'annule

8.1 — Un DDL au milieu d'une transaction

Théorie : `Theorie_03_SQL.md`, section 8.

Objectif : vérifier ce que DuckDB fait d'un `CREATE TABLE` glissé dans une transaction.

```
SELECT YearlyIncome FROM LABO03_CLIENT WHERE CustomerKey = 11349;

BEGIN;
UPDATE LABO03_CLIENT SET YearlyIncome = 20000 WHERE CustomerKey = 11349;
CREATE OR REPLACE TABLE LABO03_DDL (x INTEGER);
ROLLBACK;

SELECT YearlyIncome FROM LABO03_CLIENT WHERE CustomerKey = 11349;

SELECT COUNT(*) AS labo03_ddl
FROM information_schema.tables
WHERE table_name = 'LABO03_DDL';
```

Observez. Le revenu de Mindy Luo valait 10 000 $, et il les vaut toujours. `information_schema.tables`, le catalogue des tables que la norme SQL prévoit et que DuckDB comme PostgreSQL tiennent, ne connaît aucune `LABO03_DDL` : 0. Le `ROLLBACK` a tout annulé, la création de table comprise. Sur DuckDB, le DDL est transactionnel, comme sur PostgreSQL.

Portabilité — Snowflake ne fait pas de même : d'après sa documentation, un DDL y valide d'office la transaction en cours, puis s'exécute à part. Le même bloc y aurait laissé l'`UPDATE` validé et la table créée, sans rien à annuler pour le `ROLLBACK`. On y sépare donc le DDL des écritures qu'on veut pouvoir défaire, et le filet est ailleurs, dans le voyage dans le temps du Cours 11.

8.2 — Même un `DROP`

Théorie : `Theorie_03_SQL.md`, section 8.

Objectif : supprimer une table dans une transaction, puis la récupérer.

© 2026 iAlexMG. Tous droits réservés.

```sql
BEGIN;
DROP TABLE LABO03_CLIENT_BRUT;
SELECT COUNT(*) AS labo03_client_brut
FROM information_schema.tables
WHERE table_name = 'LABO03_CLIENT_BRUT';
ROLLBACK;

SELECT COUNT(*) FROM LABO03_CLIENT_BRUT;
```

Observez. 0, la table a disparu, puis elle revient avec ses 506 lignes. Tant que la transaction est ouverte, DuckDB garde de quoi tout défaire. Ce filet ne vaut que si vous avez tapé `BEGIN` avant : hors transaction, le `DROP` est validé à la seconde où il réussit.

Nettoyage

```sql
DROP VIEW IF EXISTS LABO03_CLIENT_AISE;
DROP VIEW IF EXISTS LABO03_CLIENT_TOUT;
DROP VIEW IF EXISTS LABO03_VUE_JETABLE;
DROP TABLE IF EXISTS LABO03_ENVOI;
DROP SEQUENCE IF EXISTS LABO03_ENVOI_SEQ;
DROP TABLE IF EXISTS LABO03_CLIENT;
DROP TABLE IF EXISTS LABO03_CLIENT_BRUT;
DROP TABLE IF EXISTS LABO03_CLIENT_MAJ;
DROP TABLE IF EXISTS LABO03_ESSAI_TYPE;
DROP TABLE IF EXISTS LABO03_CHECK;
DROP TABLE IF EXISTS LABO03_DDL;
```

L'ordre compte : les envois avant les clients, que leur clé étrangère protège, et la séquence après la table qui l'appelle. Dans l'autre ordre, DuckDB refuse les deux suppressions, `IF EXISTS` ou non.

Les exercices du cours et leur corrigé vivent dans `Exercices_corriges_03_SQL.md`. Faites-les après avoir parcouru les deux labos.

Et ensuite ?

Passez au labo jumeau, `Labo_03_OLTP_SQL.md` : les mêmes gestes sur PostgreSQL, où un `VARCHAR(3)` refuse `ABCD`, où le compteur d'identité se recale d'une commande, et où `MERGE` refuse une source en double. Le Cours 04 revient ensuite à la lecture, avec les fonctions de fenêtrage.

© 2026 iAlexMG. Tous droits réservés.